

Decision rules on a short scale: five polytomous items

Does the width-to-error-rate relation hold below nine items?

Magnus Johansson

2026-08-04

Table of contents

Scope	1
Misfit calibration	1
Design	1
Run	3
Achieved misfit	4
The Sidak relation at $k = 5$	4
Error rates of the p-value rules under the null	5
Detection and false positives, all four rules	5
Benjamini-Hochberg reachability	7

Scope

The width and iterations studies verified the Sidak relation $1 - w^k$ at $k = 9, 15$ and 20 items, in both item formats. Nothing below nine items was tested, and the manuscript says so in its Limitations: each item is a larger share of the total score on a short scale, so the dependence among item fit statistics is stronger, and the Sidak bound could be looser there than it is at $k = 20$.

This study fills that gap at $k = 5$ and reruns the decision-rule comparison under otherwise identical conditions.

Two things make short scales worth a separate look.

The Sidak prediction is much lower. At $k = 5$ the predicted familywise error rates are 41.0, 22.6, 4.9, 2.5 and 0.5 percent for the five widths, against 18.2 percent at $w = .99$ for $k = 20$. If positive dependence among item statistics makes the bound conservative, the effect should be most visible where the predicted rate is small.

The Benjamini-Hochberg floor stops binding. Its reachability floor is $m/\alpha - 1$, which is 399 at 20 items but only **99 at 5 items**. So BH can flag a standout item at every value of B tested here, which is not true in any other study in this series. This is the one setting where BH competes on equal terms.

Misfit calibration

Misfit severity is anchored on the infit scale, as everywhere else. ρ is not portable across item counts any more than across item formats: at 15 polytomous items $\rho = .85$ injects infit around 1.32, while at 5 items it injects only 1.20. A short scale needs stronger misfit for the same infit, because the misfitting item is a larger part of the total score it is conditioned on and therefore attenuates its own statistic more.

A scan of 80 replications at $n = 600$ gives:

ρ	injected infit, $k = 1$
0.75	1.31
0.85	1.20
0.90	1.14
0.95	1.06

$\rho = 0.75$ is used, which lands inside the 1.23 to 1.32 band that the real phq9 items reach. The study reports the achieved value so the anchoring is verifiable from the run.

Design

Five polytomous items with four response categories, locations evenly spaced between -2 and 2 logits, thresholds spread one logit around each location, and person locations with a standard deviation of 1.5. This is the [sim15](#) recipe at $k = 5$. The misfitting item is the one at 0 logits, on target relative to the person mean. Sample sizes are 250 and 600, iterations 100, 400 and 2000, and all five interval widths and all four decision rules are computed from the same bootstrap within each replication, so every comparison is paired.

```

suppressMessages({
  pkgload::load_all("../", quiet = TRUE) # internal helpers
  library(ggdist)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
NS <- c(250, 600)
BS <- c(100, 400, 2000)
RHO <- 0.75 # injects infit ~1.31 at k = 5, see the scan above
KS <- c(0L, 1L) # 0 = null (error rates), 1 = one misfitting item
REPS <- 250
NCORES <- 10
ALPHA <- 0.05
CHUNK <- 50
RES <- "infit_short_scale.rds"

LOCS <- seq(-2, 2, length.out = 5)
THR <- lapply(LOCS, function(L) L + seq(-1, 1, length.out = 3))
MIS <- which.min(abs(LOCS)) # item 3, at 0 logits
SD <- 1.5
NM <- paste0("V", seq_along(LOCS))

```

```

run_resumable <- function(res_file, grid, keys, fn,
                          ncores = NCORES, chunk_size = CHUNK,
                          extra = list()) {
  key_of <- function(g) {
    if (nrow(g) == 0L) return(character(0))
    do.call(paste, c(unname(as.list(g[keys])), sep = "|"))
  }
  acc <- if (file.exists(res_file)) readRDS(res_file) else
    c(list(res = list(), grid = grid[0, , drop = FALSE], elapsed_min = 0), extra)
  todo <- which(!(key_of(grid) %in% key_of(acc$grid)))
  if (length(todo) == 0L) {
    message("cached: ", nrow(acc$grid), " cells, nothing to run"); return(acc)
  }
  message("running ", length(todo), " new cells (", nrow(acc$grid), " cached)")
  for (s in seq(1, length(todo), by = chunk_size)) {
    idx <- todo[s:min(s + chunk_size - 1L, length(todo))]
    t0 <- Sys.time()
    out <- mclapply(idx, function(i) fn(grid[i, , drop = FALSE]), mc.cores = ncores)
    acc$res <- c(acc$res, out)
    acc$grid <- rbind(acc$grid, grid[idx, , drop = FALSE])
    acc$elapsed_min <- acc$elapsed_min + as.numeric(Sys.time() - t0, "mins")
    acc <- utils::modifyList(acc, extra)
    saveRDS(acc, res_file)
    message(" checkpoint ", nrow(acc$grid), " / ", nrow(grid),
            " (", round(acc$elapsed_min, 1), " min)")
  }
  acc
}

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

```

```

}
flag_code <- function(msq, lo, hi) ifelse(msq > hi, 2L, ifelse(msq < lo, 1L, 0L))

```

```

sim_short <- function(n, k, rho, seed) {
  set.seed(seed)
  S <- SD^2 * matrix(c(1, rho, rho, 1), 2, 2)
  th <- MASS::mvrnorm(n, mu = c(0, 0), Sigma = S)
  d <- sim_partial_score(THR, th[, 1])
  if (k > 0L) d[, MIS] <- sim_partial_score(THR[MIS], th[, 2])[, 1]
  d <- as.data.frame(d)
  names(d) <- NM
  d
}

```

Run

```

one_rep <- function(g) {
  out <- try({
    k <- g$k; n <- g$n; rep <- g$rep
    d <- sim_short(n, k, RHO, seed = 6e8 + 1e6 * k + 1e3 * n + rep)
    items <- names(d)
    obs <- RMitemInfit(d, output = "dataframe")
    msq <- obs$Infit_MSQ[match(items, obs$Item)]
    hd <- list(); pv <- list()
    for (j in seq_along(BS)) {
      co <- try(RMitemInfitCutoff(d, iterations = BS[j], parallel = FALSE,
                                seed = rep * 100 + j, verbose = FALSE),
              silent = TRUE)
      if (inherits(co, "try-error")) next
      hd[[length(hd) + 1L]] <- do.call(rbind, lapply(WIDTHS, function(w) {
        b <- bounds_at(co$results, w, items)
        data.frame(B = BS[j], w = w, item = seq_along(items),
                  code = flag_code(msq, b[1, ], b[2, ]))
      })))
      fw <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
                  alpha = ALPHA, output = "dataframe")), silent = TRUE)
      bh <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
                  alpha = ALPHA, output = "dataframe")), silent = TRUE)
      if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
        i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
        pv[[length(pv) + 1L]] <- data.frame(
          B = BS[j], item = seq_along(items),
          msq = msq,
          p_marg = fw$p_infit[i1], padj_fwer = fw$padj_infit[i1],
          padj_bh = bh$padj_infit[i2])
      }
    }
    list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

grid <- expand.grid(rep = 1:REPS, n = NS, k = KS)[, c("k", "n", "rep")]
grid <- grid[order(grid$k, grid$n), ]
rownames(grid) <- NULL
cat("cells to run:", nrow(grid), "\n")

```

cells to run: 1000

```
s <- run_resumable(RES, grid, c("k", "n", "rep"), one_rep,
  extra = list(locs = LOCS, mis = MIS, rho = RH0,
    widths = WIDTHS, bs = BS, alpha = ALPHA))

bind_part <- function(part) {
  do.call(rbind, lapply(seq_len(nrow(s$grid)), function(i) {
    r <- s$res[[i]][[part]]
    if (is.null(r)) return(NULL)
    g <- s$grid[i, c("k", "n", "rep"), drop = FALSE]
    rownames(g) <- NULL
    cbind(g, r)
  })))
}
H <- bind_part("hdc") |>
  mutate(is_mis = k > 0 & item == MIS, flag = as.integer(code > 0),
    w = sprintf("%.3f", w))
P <- bind_part("pval") |> mutate(is_mis = k > 0 & item == MIS)
cat(nrow(s$grid), "cells complete |", round(s$elapsed_min, 1), "min |",
  "failed:", sum(vapply(s$res, is.null, logical(1))), "\n")
```

```
1000 cells complete | 73 min | failed: 0
```

Achieved misfit

```
P |>
  filter(is_mis, B == 2000) |>
  summarise(infit = round(mean(msq), 3), sd = round(sd(msq), 3), .by = n)
```

n	infit	sd
250	1.33	0.095
600	1.32	0.059

The Sidak relation at $k = 5$

Predicted familywise error rate, $1 - w^5$, against the observed rate under the null. The interval rule flags an item when its observed infit falls outside the band, so under the null any flag is a false positive.

```
pred <- data.frame(w = sprintf("%.3f", WIDTHS),
  predicted = round(100 * (1 - WIDTHS^5), 1))
H |>
  filter(k == 0) |>
  summarise(any = as.integer(sum(flag) > 0), .by = c(w, B, n, rep)) |>
  summarise(observed = round(100 * mean(any), 1), .by = c(w, B, n)) |>
  pivot_wider(names_from = n, values_from = observed, names_prefix = "n = ") |>
  left_join(pred, by = "w") |>
  arrange(w, B) |>
  relocate(predicted, .after = w)
```

w	predicted	B	n = 250	n = 600
0.900	41.0	100	48.0	43.2
0.900	41.0	400	43.2	38.8
0.900	41.0	2000	43.2	35.6
0.950	22.6	100	29.4	26.4
0.950	22.6	400	26.0	22.8
0.950	22.6	2000	25.6	21.6

w	predicted	B	n = 250	n = 600
0.990	4.9	100	12.5	9.2
0.990	4.9	400	8.0	6.4
0.990	4.9	2000	8.0	4.8
0.995	2.5	100	9.3	7.2
0.995	2.5	400	6.4	4.0
0.995	2.5	2000	4.4	2.8
0.999	0.5	100	9.3	7.2
0.999	0.5	400	4.0	1.6
0.999	0.5	2000	1.2	0.8

Error rates of the p-value rules under the null

A replication is dropped when its bootstrap yields no usable p-values, which happens when the simulated statistics have no variance to studentise by. This occurred once in 1000 cells, at $n = 250$ and $B = 400$, and `dropped` counts it so the loss is visible rather than silently absorbed.

```
P |>
  filter(k == 0) |>
  summarise(usable = !all(is.na(p_marg)),
            marginal = as.integer(sum(p_marg < ALPHA, na.rm = TRUE) > 0),
            wy = as.integer(sum(padj_fwer < ALPHA, na.rm = TRUE) > 0),
            bh = as.integer(sum(padj_bh < ALPHA, na.rm = TRUE) > 0),
            .by = c(B, n, rep)) |>
  summarise(dropped = sum(!usable),
            across(c(marginal, wy, bh),
                    \(x) round(100 * mean(x[usable]), 1)),
            .by = c(B, n)) |>
  arrange(n, B)
```

B	n	dropped	marginal	wy	bh
100	250	0	23.0	6.5	4.8
400	250	1	24.1	5.6	5.2
2000	250	0	23.2	5.2	6.0
100	600	0	19.6	5.2	2.8
400	600	0	20.0	4.0	3.6
2000	600	0	20.0	4.0	3.6

Detection and false positives, all four rules

```
bind_rows(
  H |>
    filter(k > 0, w == "0.999") |>
    summarise(rule = "Interval .999",
              det = 100 * mean(flag[is_mis]),
              fp = 100 * mean(flag[!is_mis]), .by = c(n, B)),
  H |>
    filter(k > 0, w == "0.950") |>
    summarise(rule = "Interval .95",
              det = 100 * mean(flag[is_mis]),
              fp = 100 * mean(flag[!is_mis]), .by = c(n, B)),
  P |>
    filter(k > 0) |>
```

```

summarise(rule = "Westfall-Young",
  det = 100 * mean(padj_fwer[is_mis] < ALPHA, na.rm = TRUE),
  fp = 100 * mean(padj_fwer[!is_mis] < ALPHA, na.rm = TRUE), .by = c(n, B)),
P |>
  filter(k > 0) |>
  summarise(rule = "Benjamini-Hochberg",
    det = 100 * mean(padj_bh[is_mis] < ALPHA, na.rm = TRUE),
    fp = 100 * mean(padj_bh[!is_mis] < ALPHA, na.rm = TRUE), .by = c(n, B))
) |>
  mutate(across(c(det, fp), \(x) round(x, 1))) |>
  arrange(n, rule, B)

```

n	B	rule	det	fp
250	100	Benjamini-Hochberg	86.8	6.3
250	400	Benjamini-Hochberg	90.4	6.8
250	2000	Benjamini-Hochberg	91.2	6.6
250	100	Interval .95	96.0	19.9
250	400	Interval .95	97.6	16.3
250	2000	Interval .95	96.4	14.9
250	100	Interval .999	87.6	7.7
250	400	Interval .999	78.0	2.4
250	2000	Interval .999	73.2	0.7
250	100	Westfall-Young	91.2	3.9
250	400	Westfall-Young	91.2	4.0
250	2000	Westfall-Young	90.8	4.2
600	100	Benjamini-Hochberg	100.0	21.0
600	400	Benjamini-Hochberg	100.0	21.9
600	2000	Benjamini-Hochberg	100.0	22.7
600	100	Interval .95	100.0	34.5
600	400	Interval .95	100.0	34.4
600	2000	Interval .95	100.0	32.3
600	100	Interval .999	100.0	18.1
600	400	Interval .999	100.0	8.4
600	2000	Interval .999	99.6	4.6
600	100	Westfall-Young	100.0	16.8
600	400	Westfall-Young	100.0	16.6
600	2000	Westfall-Young	100.0	16.3

```

bind_rows(
  H |> filter(k > 0, w == "0.999") |>
    summarise(det = 100 * mean(flag[is_mis]), .by = c(n, B)) |>
    mutate(rule = "Interval .999"),
  P |> filter(k > 0) |>
    summarise(det = 100 * mean(padj_fwer[is_mis] < ALPHA, na.rm = TRUE), .by = c(n, B)) |>
    mutate(rule = "Westfall-Young"),
  P |> filter(k > 0) |>
    summarise(det = 100 * mean(padj_bh[is_mis] < ALPHA, na.rm = TRUE), .by = c(n, B)) |>
    mutate(rule = "Benjamini-Hochberg")
)

```

```

) |>
  ggplot(aes(factor(B), det, colour = rule, group = rule)) +
  geom_line() + geom_point(size = 2) +
  facet_wrap(~ n, labeller = label_both) +
  ylim(0, 100) +
  labs(x = "Bootstrap iterations (B)", y = "Detection (%)", colour = "Rule") +
  theme_minimal(base_size = 10)

```

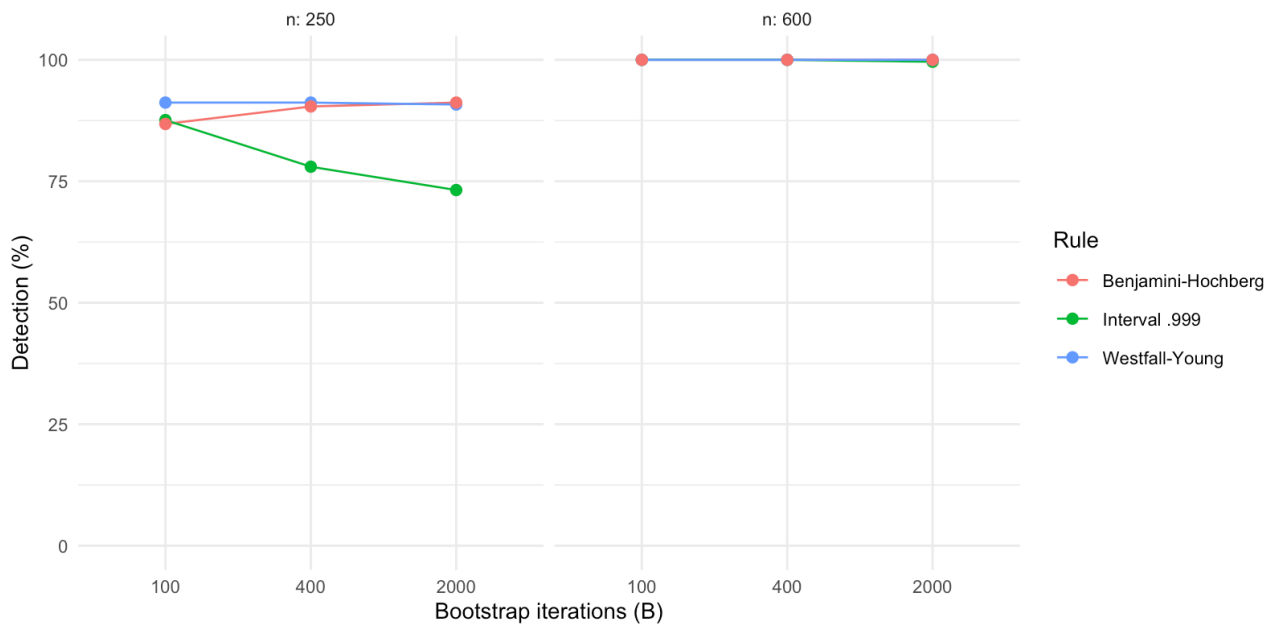


Figure 1

Benjamini-Hochberg reachability

At five items the floor is $m/\alpha - 1 = 99$ iterations, so BH is reachable at every B tested. The smallest attainable adjusted p-value is shown against alpha for reference.

```

data.frame(B = BS) |>
  mutate(floor_single = 5 / (B + 1),
         reachable = floor_single < ALPHA)

```

B	floor_single	reachable
100	0.0495050	TRUE
400	0.0124688	TRUE
2000	0.0024988	TRUE